

Put `std::monostate` in `<utility>`

- Document number: P0472R0
- Date: 2016-10-14
- Reply-to: David Sankel <dsankel@bloomberg.net>
- Audience: Library Evolution

Abstract

`std::monostate` is currently defined and available in the `<variant>` header, but its utility is not limited to variants. We propose moving `std::monostate` to `<utility>` to reduce artificial coupling of `std::monostate` clients to variants.

Motivation

`std::monostate` is a class that has exactly one value. It is default constructable and supports all the comparison operations. `std::monostate` is about as simple of a type as one could concoct. These properties turn out to be useful for writing template code.

The first use case is in testing. Does your custom `vector` or `set` make any undesirable assumptions about the types they are instantiated with? If they work properly with `std::monostate`, then probably not. `std::monostate` can be used in this way as a means to write simple test drivers.

The second use case occurs in more sophisticated template metaprogramming. The well known `std::future` class makes use of a "special" template parameter `void` to indicate that it carries no information aside from when the future is fulfilled. Using the `void` keyword to represent this situation carries a serious implementation burden due to its many strange properties. While this burden may not be a problem for standard library implementers, it would be nice to have a simpler option for the more common developer.

`std::monostate` is a much more natural way to represent "no information" than `void` is. It has exactly one value and is regular type instead of a keyword. Consider how simple the following code is:

```
template<typename ExtraInformation = std::monostate>
class Data
{
    //...
    ExtraInformation m_extraInformation;
}
```

Here we have a `Data` template which optionally carries extra information. The use of `std::monostate` in this example makes it simple in both specification and implementation to represent `Data` objects that carry no additional information.

Conclusion

`std::monostate` is a generally useful type and therefore belongs in a more appropriate header than `<variant>`. If the committee deems this a worthy course of action, wording will be provided.